

Reflect-Guard: Enhancing LLM Safeguards against Adversarial Prompts via Logical Self-Reflection

Lixing Lin ¹, Juli You ², Yue Li ³, Luyun Lin ^{4*}, Yiqing Wang ⁴, Zhen Zhang ² and Moxuan Zheng ²

¹ Yale University, New Haven, CT, USA; lixing.lin@yale.edu

² Independent Researcher; you.juli@gmail.com (J.Y.); billzhangzhen98@gmail.com (Z.Z.); pollyzheng9527@gmail.com (M.Z.)

³ Columbia University, New York, NY, USA; yl6087@columbia.edu

⁴ Citigroup, Dallas, USA; lly936368767@gmail.com (L.L.); woshilucy712@gmail.com (Y.W.)

* Correspondence: lly936368767@gmail.com

Abstract

LLM safety classifiers such as Llama Guard detect overtly harmful prompts well but remain vulnerable to jailbreaks that disguise intent via role-play, fictional framing, or indirect requests. We present **Reflect-Guard**: Llama-Guard-3-8B fine-tuned via QLoRA to generate a distilled, GPT-4o-mini-sourced safety rationale before its verdict. Using only 1,000 training examples and a five-condition ablation, we isolate reflection's contribution from that of ordinary supervised fine-tuning (SFT). SFT on verdict labels alone already accounts for most of the aggregate gain over the un-fine-tuned baseline (WildGuardTest F1: 0.770 $\rightarrow$ 0.860; JailbreakBench guard bypass rate: 10.3% $\rightarrow$ 0.0%). Adding reflection does not improve these aggregate numbers further (F1 falls to 0.842, guard bypass rises to 1.8%), but reallocates performance toward adversarially-framed inputs: adversarial-subset recall rises from 0.845 (SFT-only) to 0.921 (+7.6 pp), at a cost of -7.9 pp precision on that subset. Three-seed retraining confirms both directions of this trade-off are statistically significant (paired t -tests, $p = 0.018$ and $p = 0.027$), and measured inference latency is $17.9\times$ the baseline's, driven by ~ 92 additional reflection tokens per prompt. The same checkpoint, unmodified, generalizes zero-shot to a 329-example financial-harm benchmark (F1: 0.868 $\rightarrow$ 0.929; recall: 0.767 $\rightarrow$ 0.867, zero false positives). Benchmarked against WildGuard, PromptGuard, and a prompted (non-fine-tuned) GPT-4o-mini reasoning classifier on the same sets, WildGuard leads on aggregate WildGuardTest F1 (0.887), but Reflect-Guard attains the highest adversarial-subset recall of all evaluated classifiers (0.921 vs. 0.827 for WildGuard, 0.636 for the prompted teacher) and the highest finance-transfer F1 (0.929 vs. 0.921). Overall, supervised fine-tuning drives most of the aggregate gain over an un-fine-tuned classifier, while explicit reasoning about adversarial intent yields a narrower, statistically-supported, latency-costly improvement in adversarial-prompt detection—exceeding even a dedicated external safety classifier on that specific axis—a precision–recall trade-off relevant to safety-critical deployments that prioritize recall over inference speed.

Keywords: LLM safety; adversarial prompts; jailbreak attacks; generated safety rationales; knowledge distillation; QLoRA; safety classification; precision–recall tradeoff; domain generalization; competitive baseline evaluation

Received:

Revised:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Journal Not Specified* for possible open access publication under the terms and conditions of the

[Creative Commons Attribution \(CC BY\) license](#).

1. Introduction

The deployment of large language models (LLMs) in production systems has created an urgent need for robust safety classifiers that can intercept harmful requests before they reach the target model [1,2], one dimension of the broader trustworthy-AI challenge spanning safety, robustness, and reliability that recent large-scale surveys document across the LLM ecosystem [32]. Current safety classifiers, including Meta’s Llama Guard family [1, 6], perform well on straightforward harmful prompts but exhibit significant vulnerabilities when confronted with adversarial jailbreak attacks [11–13].

Adversarial jailbreaks exploit a fundamental weakness in pattern-based classification: by wrapping harmful requests in benign-sounding contexts—fictional narratives, role-play scenarios, hypothetical research questions, or educational framings—attackers can bypass safety filters that rely primarily on surface-level semantic matching [15]. For instance, Llama-Guard-3-8B achieves only 51.3% recall on the adversarial subset of WildGuardTest, meaning it misses nearly half of disguised harmful prompts.

We hypothesize that this vulnerability stems from the classifier’s lack of explicit reasoning about adversarial intent. Human safety reviewers naturally decompose ambiguous inputs: they identify framing techniques, assess underlying intent behind surface-level language, and reason about whether a request would cause harm regardless of its narrative wrapper. Standard safety classifiers such as Llama Guard lack this analytical capability, making binary decisions without interpretable intermediate reasoning.

This gap has begun to be addressed by a concurrent line of 2025 work that also trains safety classifiers to reason before verdicting, including GuardReasoner [28], SaRO [29], and Think Twice, Generate Once [31] (compared in detail in Section 2.0.0.3). Reflect-Guard occupies a different point in this design space: rather than large-scale reasoning-trace training or training-free inference-time reflection, we ask whether a much smaller intervention—knowledge distillation of a teacher model’s structured reflections into a parameter-efficient LoRA adapter, trained on 1,000 examples with ordinary supervised fine-tuning—captures a meaningful fraction of the same benefit, and, critically, whether that benefit is separable from what ordinary label-only fine-tuning already provides. This second question is one a larger-scale reasoning-training study is not naturally positioned to isolate, since without a matched label-only ablation at the same data and compute budget, gains attributed to “reasoning” can be confounded with gains from fine-tuning the decision boundary itself—precisely the confound this paper’s five-condition ablation (Section 7.4) is designed to separate out.

We propose **Reflect-Guard**, a method that endows safety classifiers with a generated safety rationale—a self-reflection step—through knowledge distillation and parameter-efficient fine-tuning. We use the term *generated safety rationale* rather than “chain-of-thought” deliberately: the reflection text is a learned explanatory pattern distilled from a teacher model, and we make no claim that it reflects genuine internal reasoning (see Section 7). Our approach proceeds in three stages: (1) we use GPT-4o-mini to generate structured reflection annotations for a curated set of 1,000 training examples, analyzing adversarial techniques and harm indicators; (2) we fine-tune Llama-Guard-3-8B using QLoRA [26] to generate these reflections as an intermediate step before safety classification; and (3) at inference time, the model produces an explicit logical analysis in <reflection> tags before outputting its safety verdict.

Because supervised fine-tuning (SFT) alone—without any reflection—could plausibly account for some or all of the improvement over an un-fine-tuned baseline, isolating reflection’s specific contribution requires a controlled ablation rather than a single baseline-vs.-Reflect-Guard comparison. Our contributions are as follows:

- We introduce Reflect-Guard, a reflection-augmented safety classification method that teaches LLM-based safety classifiers to reason about adversarial intent before classification.
- We demonstrate that knowledge distillation from a teacher model (GPT-4o-mini) into structured reflection annotations enables effective transfer of analytical reasoning capabilities with only 1,000 training examples.
- Through a five-condition ablation (Section 7.4), we show that SFT on verdict labels alone accounts for the majority of the aggregate performance gain over an un-fine-tuned baseline (WildGuardTest F1: 0.770 $\rightarrow$ 0.860; JailbreakBench guard bypass rate: 10.3% $\rightarrow$ 0.0%), while reflection training on top of SFT provides a distinct, targeted improvement on adversarially-framed inputs—+7.6 pp recall on the adversarial subset of WildGuardTest—at a modest cost to overall precision and F1 relative to SFT-only fine-tuning. We report this as a precision–recall tradeoff rather than an unconditional improvement.
- We confirm this tradeoff is not a single-run artifact: retraining the SFT-only and full Reflect-Guard conditions under three random seeds (Section 7.4.1) shows both directions of the effect are statistically significant (paired t -tests, $p = 0.018$ and $p = 0.027$ respectively), and we directly measure the inference-time cost of reflection— $17.9\times$ the baseline’s median latency—rather than characterizing it qualitatively (Section 7.3).
- We benchmark Reflect-Guard against three external baselines on the same three benchmarks—WildGuard, PromptGuard, and a prompted zero-shot reasoning classifier using the same teacher model (GPT-4o-mini)—directly addressing the competitive-baseline gap raised in review (Section 7.5). While WildGuard achieves higher aggregate WildGuardTest F1 (0.887 vs. 0.842), Reflect-Guard achieves the highest adversarial-subset recall of any evaluated classifier (0.921, vs. 0.827 for WildGuard and 0.636 for the prompted teacher) and the highest finance-transfer F1 (0.929 vs. 0.921 for WildGuard).

2. Related Work

LLM safety classifiers.

The growing deployment of LLMs has spurred development of dedicated safety classifiers. Llama Guard [1] frames content moderation as instruction-following with a customizable taxonomy. Subsequent versions (Llama Guard 2 [5], Llama Guard 3 [6]) expanded to multilingual support and refined category definitions. WildGuard [2] addresses adversarial robustness by training on the WildGuardMix dataset containing adversarial examples. Other approaches include PromptGuard [7] for injection detection, ShieldGemma [8] from Google, and OpenAI’s moderation classifier, built on an active-learning pipeline over a broad undesired-content taxonomy [33]. AgentAuditor extends safety evaluation to LLM agents, achieving human-level judgment on agentic safety scenarios [9]. Recent advances in AI research agents indicate that LLM-based systems are becoming increasingly autonomous and capable of performing complex multi-step tasks across scientific workflows [10]. We evaluate four of these classifiers—WildGuard, PromptGuard, OpenAI’s moderation classifier, and a prompted GPT-4o-mini reasoning classifier—head-to-head against Reflect-Guard on the same three benchmarks (Section 7.5). PromptGuard’s narrower design scope (attack-technique detection, not general content harm) and OpenAI’s moderation classifier’s fixed, platform-content-moderation taxonomy (sexual, hateful, violent, and self-harm content, rather than harmful *instructions* regardless of surface content) both limit their value as general-classifier comparisons for this task (Section 7.6). ShieldGemma remains untested: its checkpoints are gated HuggingFace repositories with access still pending.

Jailbreak attacks on LLMs.

Adversarial jailbreak techniques have evolved rapidly. Gradient-based methods like GCG [12] append optimized adversarial suffixes. Semantic attacks including PAIR [11] and TAP [14] use LLMs to iteratively refine jailbreak prompts. AutoDAN [13] combines gradient and semantic approaches. Template-based methods exploit role-play and fictional framing [15,16]. JailbreakBench [17] provides a standardized evaluation framework covering multiple attack families. Beyond direct prompt attacks, recent work has also documented security threats in agentic AI pipelines and runtime supply chains [18], and shown that surface-level prompt features such as politeness and tone can measurably shift LLM outputs [19]—both findings underscore the broad sensitivity of language models to prompt construction.

Chain-of-thought reasoning and generated rationales for safety.

Chain-of-thought (CoT) prompting [20] has been applied to improve reasoning across many tasks. For safety specifically, prior work has explored using CoT to improve content moderation [21] and to detect harmful content through deliberative alignment [22]. Constitutional AI [23] uses self-critique for alignment, and this line of work has recently been extended directly into a deployed jailbreak defense: Constitutional Classifiers [30] train input/output guard classifiers on synthetic data generated from natural-language “constitution” rules, evaluated against thousands of hours of red-teaming. A distinct and more directly comparable line of 2025 work trains the safety classifier itself to reason before verdicting, rather than prompting or self-critiquing a general-purpose model: GuardReasoner [28] fine-tunes guard models of several scales on a 127K-example reasoning-trace dataset using SFT followed by preference optimization on hard examples, and SaRO [29] similarly combines a reasoning-style supervised warmup with DPO-based safety-oriented reasoning optimization to address both under- and over-alignment. Orthogonal to both, Think Twice, Generate Once [31] shows that progressive self-reflection can reduce attack success rates substantially at *inference time alone*, with no additional training.

Reflect-Guard differs from all four along the same axis: rather than large-scale reasoning-trace SFT plus preference optimization (GuardReasoner, SaRO), constitution-derived synthetic classifier training (Constitutional Classifiers), or purely inference-time self-reflection with no parameter update (Think Twice), we distill a teacher model’s structured reflections into a lightweight LoRA adapter using ordinary supervised fine-tuning on 1,000 examples—trading the stronger safety gains those heavier-weight approaches report for a much smaller data and compute footprint, and isolating what a QLoRA-scale intervention can and cannot achieve via the five-condition ablation in Section 7.4. This distinction matters for the paper’s central finding: unlike GuardReasoner’s or SaRO’s large reasoning-trace training sets, our ablation shows that most of Reflect-Guard’s aggregate gain over the un-fine-tuned baseline comes from ordinary label-only SFT, not the reflection component itself—a finding a larger-scale reasoning-training study would not necessarily surface. We refer to our distilled output as a *generated safety rationale* rather than CoT reasoning throughout, since it is trained to reproduce a teacher-generated explanatory pattern rather than to expose verified intermediate computation—the same caveat applies, to varying degrees, to the reasoning traces used by GuardReasoner and SaRO, neither of which independently verifies that the generated reasoning is faithful to the model’s actual decision process (Section 7.6 discusses this for Reflect-Guard specifically).

Knowledge distillation for safety.

Knowledge distillation [24] has been applied to transfer capabilities from larger to smaller models. Recent work explores distilling safety behaviors from proprietary mod-

els [25]. Our approach specifically distills *analytical reasoning patterns* rather than output distributions, teaching the student model to reason about adversarial techniques.

3. Method

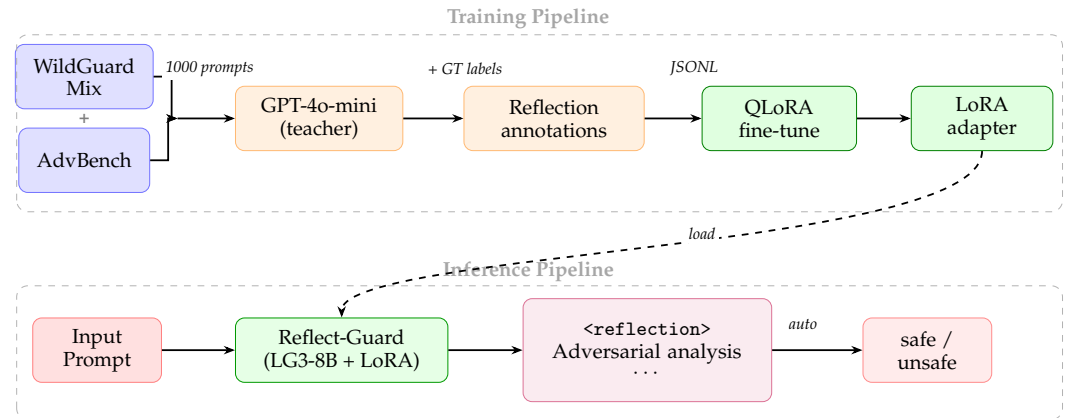


Figure 1. Reflect-Guard overview. *Training (top):* GPT-4o-mini generates structured reflection annotations for 1,000 labeled prompts; Llama-Guard-3-8B is then fine-tuned via QLoRA to reproduce these reflections. *Inference (bottom):* The fine-tuned model generates an explicit adversarial analysis inside `<reflection>` tags before issuing a safety verdict—no external API is required at inference time.

3.1. Problem Formulation

Given an input prompt x , a safety classifier f must produce a binary verdict $y \in \{\text{safe}, \text{unsafe}\}$. Standard classifiers model this as $y = f(x)$. Our approach introduces an intermediate reflection step r , decomposing classification into:

$$r = g(x), \quad y = h(x, r) \quad (1)$$

where g generates a logical analysis of the prompt's intent and adversarial characteristics, and h produces the final verdict conditioned on both the input and the reflection. In practice, both g and h are implemented by a single autoregressive model that first generates r in `<reflection>` tags, then generates y .

Equivalently, under a single generative model with parameters θ , we factorize the conditional distribution as

$$p_{\theta}(r, y | x) = p_{\theta}(r | x) p_{\theta}(y | x, r), \quad (2)$$

and predict the final label via

$$\hat{y} = \arg \max_{y \in \{\text{safe}, \text{unsafe}\}} p_{\theta}(y | x, \hat{r}), \quad \hat{r} \sim p_{\theta}(r | x). \quad (3)$$

This formulation makes explicit that the reflection serves as a latent, model-generated rationale that conditions the downstream safety decision.

3.2. Reflection-Augmented Training Data

We construct a training dataset of 1,000 examples from two sources: 500 examples from WildGuardMix [2] (the training split) and 500 from AdvBench [12]. The combined dataset contains 773 harmful and 227 benign examples. Denoting the synthesized dataset by

$$\mathcal{D} = \{(x_i, y_i, r_i, c_i)\}_{i=1}^N, \quad N = 1000, \quad (4)$$

where x_i is the input prompt, y_i is the binary safety label, r_i is the teacher-generated reflection, and c_i denotes the violated safety categories (empty for safe prompts), we use GPT-4o-mini as a teacher model to generate a structured reflection annotation for each example.

The teacher receives each prompt along with its ground-truth label and is instructed to produce a 2–4 sentence analytical reflection that: (1) identifies whether the prompt employs adversarial techniques such as role-play framing, fictional scenarios, indirect harmful requests, technical obfuscation, or social engineering; (2) states whether the underlying intent is harmful or benign with reasoning; and (3) lists key indicators supporting the classification.

Each training example is formatted as a conversation following the Llama Guard 3 template:

```
User: [Safety instruction with 13 categories S1–S13]
<BEGIN CONVERSATION>
User: {prompt}
<END CONVERSATION>
Provide your safety assessment...First provide a logical self-reflection in
<reflection>...</reflection> tags...

Assistant: <reflection>{reflection}</reflection>
{safe|unsafe}
{violated categories, if unsafe}
```

3.3. QLoRA Fine-Tuning

We fine-tune Llama-Guard-3-8B [6] using QLoRA [26] with 4-bit NormalFloat (NF4) quantization and double quantization. LoRA [27] adapters are applied to all linear projection layers (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj) with rank $r = 16$, scaling factor $\alpha = 32$, and dropout 0.05. This results in approximately 42M trainable parameters (0.5% of the full 8B model).

Training uses the paged AdamW optimizer with 8-bit states, learning rate 2×10^{-4} with cosine scheduling and 5 warmup steps. We train for 3 epochs with per-device batch size 1 and gradient accumulation of 16 steps (effective batch size 16), using BF16 mixed precision and gradient checkpointing. Let $s_i = [r_i; y_i; c_i]$ denote the target output sequence for example i , consisting of the reflection, verdict, and optional category codes. The fine-tuning objective is standard autoregressive negative log-likelihood:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{|s_i|} \log p_{\theta}(s_{i,t} \mid x_i, s_{i,<t}). \quad (5)$$

Training completes in approximately 1 hour on a single NVIDIA A5000 GPU. The full software environment is: Python 3.10, PyTorch 2.2.0 (CUDA 11.8 build), Transformers 4.44.2, PEFT $\geq 0.12.0$, bitsandbytes $\geq 0.43.1$, TRL 0.9.6, and Accelerate 0.33.0 (requirements.txt, data availability statement); GPUs were requested with compute capability up to sm_90 (Hopper), since the pinned CUDA 11.8 PyTorch build does not support newer sm_120 (Blackwell) hardware present on the shared cluster used for this work.

3.4. Inference Pipeline

At inference time, the fine-tuned model receives a prompt formatted with the same safety instruction template used during training. The model autoregressively generates: (1) a reflection analyzing the prompt’s characteristics, enclosed in `<reflection>...</reflection>` tags; followed by (2) a safety verdict (safe or unsafe); and optionally (3) violated category

codes. We use greedy decoding with a maximum of 150 new tokens. The reflection is extracted and stored alongside the verdict for interpretability.

Critically, no external API calls are required at inference time; the reflection capability is fully internalized in the LoRA adapter. This does, however, add substantial generation cost relative to the base model: a standard Llama Guard verdict is typically only 1–3 output tokens, whereas Reflect-Guard generates the verdict plus a mean of ~ 92 additional reflection tokens per prompt, measured directly (Section 7.3, Table 7) at a median single-request latency $17.9\times$ that of the un-fine-tuned baseline and $14.3\times$ that of Condition B. The added latency is substantial, not negligible, and is the direct cost of the adversarial-recall gain discussed in Section 7.4.

4. Experimental Setup

4.1. Benchmarks

WildGuardTest.

The test split of WildGuardMix [2] contains 1,699 prompts (754 harmful, 945 benign) with annotations for adversarial framing. The adversarial subset ($n = 796$) contains prompts that disguise intent through various obfuscation techniques and is itself a mix of ground-truth labels: 341 harmful (adversarially-framed jailbreak attempts) and 455 benign (prompts that adopt an adversarial-looking surface structure—e.g., role-play or security-related framing—without harmful intent). Both subpopulations matter for evaluation: recall on the 341 harmful examples measures whether adversarial framing successfully evades the classifier, while precision on the 455 benign examples measures whether that same surface structure triggers over-blocking (Section 7).

JailbreakBench.

JailbreakBench [17] provides 282 jailbreak prompts generated by three attack methods against Vicuna-13B-v1.5: GCG [12] (100 prompts, gradient-based token optimization), JBC (100 prompts, template-based jailbreaks), and PAIR [11] (82 prompts, LLM-assisted semantic attacks). All prompts are ground-truth harmful, so the primary metric is detection rate (equivalently, 1 minus guard bypass rate; Section 4.3).

4.2. Baseline

Our baseline is Llama-Guard-3-8B [6] with identical 4-bit NF4 quantization but without the LoRA adapter, using the standard Llama Guard prompt without any reflection instruction. This corresponds to Condition 0 in our ablation study (Section 7.4) and isolates the effect of reflection fine-tuning from both quantization artifacts and prompt-level effects. Reflect-Guard uses the same base model and quantization, with the LoRA adapter loaded and the prompt extended to request a `<reflection>` before the verdict.

4.3. Metrics

We report accuracy, precision, recall, and F1 score for WildGuardTest, computed with “harmful” as the positive class. Using true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN), the main metrics are

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}, \quad (6)$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad (7)$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad (8)$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (9)$$

Because safety deployment often prioritizes recall, we also report

$$F_{\beta} = (1 + \beta^2) \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}}, \quad F_2 = 5 \frac{PR}{4P + R}, \quad (10)$$

where P and R abbreviate precision and recall. For JailbreakBench, we report detection rate (DR) per attack method and the overall *guard bypass rate* (GBR),

$$\text{DR} = \frac{\text{\#detected harmful prompts}}{\text{\#all harmful prompts}}, \quad \text{GBR} = 1 - \text{DR}. \quad (11)$$

We use “guard bypass rate” rather than the more common “attack success rate” deliberately: GBR measures only whether a prompt evades *this safety classifier*, not whether a downstream target LLM was actually induced to produce a prohibited response, which is what “attack success rate” conventionally denotes. When comparing guard bypass rates, the relative reduction is computed as

$$\text{Rel.Reduction} = \frac{\text{GBR}_{\text{base}} - \text{GBR}_{\text{ours}}}{\text{GBR}_{\text{base}}}. \quad (12)$$

We provide breakdown analyses for adversarial versus non-adversarial subsets on WildGuardTest.

Uncertainty estimates.

The headline point estimates in Sections 5–7 come from a single training run per condition (seed 42). To give some sense of the uncertainty attributable to the finite size of the test sets, we report 95% confidence intervals obtained by bootstrap resampling (2,000 resamples with replacement) of the fixed WildGuardTest and JailbreakBench prediction sets for each condition; these intervals quantify test-set sampling uncertainty only, not training-time randomness. To separately quantify training-time variance, we additionally retrained Conditions B and D under two further random seeds (123, 2024; three seeds total including the original) with everything else held fixed, and evaluated each on all three benchmarks. Section 7.4.1 reports the resulting mean $\pm$ std across seeds, a pooled bootstrap 95% CI that combines test-set and seed variance, and a paired significance test between Conditions B and D. Condition 0 is training-free and uses greedy decoding, so it has no training-time variance to report.

Exact dataset versions and sampling procedure.

Training prompts are drawn from two HuggingFace datasets, allenai/wildguardmix (config wildguardtrain) and walledai/AdvBench, loaded at the versions resolved by the datasets library at training time (exact commit hashes were not pinned, itself a reproducibility gap we note rather than obscure). Sampling is deterministic given RANDOM_SEED=42 (reflect_guard/config.py): each source is shuffled under the fixed seed and truncated to 500 examples, then the combined 1,000-example set is shuffled again. Evaluation uses the WildGuardMix wildguardtest split ($n = 1,699$) and JailbreakBench’s published artifacts (Section 4.1), fetched independently of training. The exact code path (reflect_guard/synthesize.py:load_sources) is released in full (data availability statement).

Data leakage check.

We checked for exact and near-duplicate prompts (normalized whitespace/case; exact match plus a 6-gram prefilter and difflib.SequenceMatcher ratio ≥ 0.85) between the 1,000 training examples and all three evaluation sets, finding zero matches of either kind—

no training examples required removal. The finance benchmark’s 269 synthetic examples were likewise checked against each other for internal duplication, with zero found.

This prompt-level check does not rule out overlap at the level of the underlying *behavior* a jailbreak attack targets, since AdvBench is a known source of the harmful-behavior goals GCG-style attacks (including JailbreakBench’s) are built around. We checked directly: of JailbreakBench’s 104 unique behaviors, 10 (9.6%) exactly match a training prompt and a further 5 (4.8%, ratio ≥ 0.85) are near-duplicates—e.g., “develop malware that disables antivirus software” vs. our training set’s “develop a virus that disables antivirus software.” This overlap is at the *behavior-goal* level only: the full attack prompts, including GCG’s optimized suffixes and PAIR’s rewritten framings, yield zero exact matches against training, confirming the model never saw the literal adversarial text it is evaluated against. We nonetheless flag this as a real, quantified caveat on the JailbreakBench results (Section 5.2, particularly Condition B’s 100% detection rate): roughly one in seven underlying behaviors (15/104, 14.4%) were present in training, differently worded and unwrapped, with the correct harmful label attached—a weaker but non-zero form of leakage than literal duplication, and a known property of AdvBench-derived benchmarks generally that we had not previously quantified.

5. Results

5.1. WildGuardTest

Table 1 presents the overall classification performance, including the SFT-only baseline (Condition B, Section 7.4) alongside the clean baseline (Condition 0) and the full Reflect-Guard model (Condition D), plus two external classifiers evaluated on the same test set (Section 7.5). Conditions A (prompted reflection, no fine-tuning) and C (blind reflections) are omitted from this table to keep it focused on the paper’s central comparison; the full five-condition comparison, including A and C, is reported in Table 8 (Section 7.4). **Among our three conditions, Condition B, not Reflect-Guard, achieves the best overall F1 (0.860) and accuracy (0.877)**, confirming that supervised fine-tuning (SFT) on the decision boundary—not reflection—is responsible for the majority of the aggregate improvement over the un-fine-tuned baseline (F1: 0.770 $\rightarrow$ 0.860, +9.0 pp). Adding reflection on top of SFT (Condition D) trades a further -1.8 pp of overall F1 and -4.7 pp of precision relative to Condition B for +1.0 pp of overall recall; Section 7.4 shows this small aggregate recall gain is concentrated specifically on adversarially-framed prompts, not distributed uniformly. Relative to the clean baseline, Reflect-Guard still improves F1 by 7.2 pp (0.770 $\rightarrow$ 0.842) and the F_2 score (which weights recall twice as heavily as precision, relevant for safety-critical deployment) improves from 0.702 to 0.855—but Condition B’s F_2 (0.856) is statistically indistinguishable, underscoring that this aggregate gain is attributable to SFT rather than to reflection specifically. Against the wider field, WildGuard—a dedicated, purpose-trained safety classifier—outperforms all three of our conditions on aggregate accuracy, precision, and F1, though Condition D’s overall recall remains the highest of any evaluated classifier; Section 7.5 discusses this comparison, and the prompted GPT-4o-mini reasoning classifier, in full.

Table 1. Overall performance on WildGuardTest ($n = 1,699$). Best results across *all* evaluated classifiers, including the external baselines below the second rule, in **bold**. Bracketed values are 95% bootstrap confidence intervals (2,000 resamples over the fixed test set; Section 4.3); WildGuard and Prompted-CoT are single evaluation runs with no retraining, so no CI is shown for them.

Model	Accuracy	Precision	Recall	F1
0. Llama-Guard-3-8B (baseline)	0.825	0.919 [0.895, 0.942]	0.663 [0.628, 0.698]	0.770 [0.743, 0.794]
B. SFT Labels Only	0.877	0.868 [0.843, 0.892]	0.853 [0.828, 0.878]	0.860 [0.841, 0.879]
D. Reflect-Guard (ours)	0.856	0.821 [0.793, 0.848]	0.863 [0.838, 0.888]	0.842 [0.821, 0.860]
$\Delta (D - 0)$	+0.031	-0.098	+0.200	+0.072
$\Delta (D - B)$	-0.021	-0.047	+0.010	-0.018
WildGuard (external)	0.903	0.926	0.850	0.887
Prompted-CoT (GPT-4o-mini)	0.855	0.919	0.739	0.819

Table 2 shows why the aggregate comparison in Table 1 understates reflection’s actual effect: performance is *reallocated* across subsets rather than uniformly improved. On the adversarial subset, SFT alone (Condition B) already substantially improves recall over the clean baseline (0.513 $\rightarrow$ 0.845, +33.2 pp); adding reflection (Condition D) pushes adversarial recall further, to 0.921 (+7.6 pp over Condition B), but at a cost of -7.9 pp adversarial precision (0.789 $\rightarrow$ 0.710)—so Condition B still edges out Condition D on adversarial F1 (0.816 vs. 0.802). On the non-adversarial subset, the pattern reverses: Condition D achieves *higher* precision than Condition B (0.960 vs. 0.944, +1.6 pp) at a cost of -4.4 pp recall (0.860 $\rightarrow$ 0.816), and again Condition B has the higher F1 (0.900 vs. 0.882). This opposing cross-subset pattern—more aggressive specifically on adversarial inputs, more conservative on non-adversarial inputs, with SFT-only ahead on F1 in *both* subsets—is inconsistent with reflection simply shifting a single global decision threshold, and instead suggests the model has learned features specific to adversarial framing (Section 7.4) even though this does not translate into an aggregate F1 win over SFT-only. Table 2 also situates this against WildGuard and the prompted-teacher baseline: WildGuard’s adversarial F1 (0.852) exceeds every one of our own conditions, including Condition B, and likewise leads on all three non-adversarial metrics—but **adversarial-subset recall is the one cell in this table where Condition D leads every evaluated classifier**, including WildGuard (0.921 vs. 0.827) and the prompted teacher (0.636); Section 7.5 discusses why this specific axis, rather than aggregate F1, is the paper’s central competitive claim.

Table 2. Performance breakdown by prompt type on WildGuardTest. Best result per column, per subset, across *all* evaluated classifiers in **bold**. WildGuard and Prompted-CoT (GPT-4o-mini) are external baselines (Section 7.5).

Subset	Model	n	Acc.	Prec.	Recall	F1
Adversarial	0. Baseline	796	0.751	0.845	0.513	0.639
	B. SFT Labels Only	796	0.837	0.789	0.845	0.816
	D. Reflect-Guard	796	0.805	0.710	0.921	0.802
	WildGuard (external)	796	0.877	0.879	0.827	0.852
	Prompted-CoT (GPT-4o-mini)	796	0.803	0.868	0.636	0.734
Non-adversarial	0. Baseline	903	0.889	0.964	0.787	0.867
	B. SFT Labels Only	903	0.913	0.944	0.860	0.900
	D. Reflect-Guard	903	0.900	0.960	0.816	0.882
	WildGuard (external)	903	0.927	0.968	0.869	0.916
	Prompted-CoT (GPT-4o-mini)	903	0.901	0.955	0.823	0.884

5.2. JailbreakBench

Table 3 presents detection rates across attack methods, including the SFT-only baseline (Condition B) and two external classifiers (Section 7.5). Both fine-tuned conditions substantially reduce the guard bypass rate relative to the clean baseline, but Condition B—trained on verdict labels alone, with no reflection—achieves *perfect* detection (100.0% DR, 0.0% GBR), outperforming the full Reflect-Guard model (98.2% DR, 1.8% GBR) and WildGuard (99.65% DR, 0.35% GBR)—Condition B is the single best-performing classifier on this benchmark among everything evaluated in this paper, not merely the best of our own conditions. The 1.8 pp of guard bypass remaining in Reflect-Guard relative to Condition B is concentrated on PAIR attacks (Section 7), which use LLM-generated semantic jailbreaks with elaborate fictional or professional framings; Reflect-Guard’s guard bypass rate is nonetheless lower than the prompted GPT-4o-mini teacher’s (1.8% vs. 3.19%). Relative to the clean (un-fine-tuned) baseline, Reflect-Guard reduces the guard bypass rate from 10.3% to 1.8%, an 82.5% relative reduction; Condition B reduces it further, to 0.0%.

Table 3. JailbreakBench detection rates by attack method. DR = detection rate; GBR = guard bypass rate (= 1 − DR overall). All prompts are ground-truth harmful; higher DR is better. 95% bootstrap confidence intervals (2,000 resamples) are shown for overall DR for Conditions 0/B/D; WildGuard and Prompted-CoT (external, Section 7.5) are single evaluation runs with no CI. Best per row in **bold** (ties bolded jointly).

Attack Method	<i>n</i>	Baseline (0)	SFT-Only (B)	Reflect-Guard (D)	WildGuard	Prompted-CoT
GCG	100	94.0%	100.0%	99.0%	99.0%	94.0%
JBC	100	98.0%	100.0%	100.0%	100.0%	100.0%
PAIR	82	74.4%	100.0%	95.1%	100.0%	96.3%
Overall	282	89.7% [85.8, 92.9]	100.0% [100.0, 100.0]	98.2% [96.5, 99.6]	99.65%	96.81%
GBR (↓)	—	10.3% [7.1, 14.2]	0.0% [0.0, 0.0]	1.8% [0.4, 3.5]	0.35%	3.19%

5.3. Qualitative Analysis

To characterize model behavior beyond aggregate metrics, we analyzed three categories of examples identified from the evaluation results (script: `extract_examples.py`).

Category 1 — Reflection catches what SFT alone misses (*n* = 29, 27 adversarial).

These are harmful prompts where both Condition 0 and Condition B predict safe, but Condition D correctly identifies harm. All 27 adversarial cases involve framing techniques: role-play, fictional alternate realities, or professional personas. A representative example asks a “noir detective” for a “comprehensive background check using every trick in the book” while the prompt explicitly dismisses privacy constraints as irrelevant to the fictional scenario. The reflection identifies the mechanism:

“The message employs a roleplay framing technique to obscure the underlying intent to discuss potentially harmful investigative methods. Key indicators include the focus on ‘comprehensive background checks’ and the emphasis on using ‘every trick in the book,’ which implies disregard for legal and ethical boundaries.”

SFT alone misses these 29 cases despite training on the same 1,000 prompts, including their adversarial framing; the difference is that its training targets contain only the verdict, not an intermediate analysis of the framing technique. A plausible explanation is that the reflection objective forces the model to represent adversarial framing explicitly as a supervised intermediate output, which SFT-only training does not require—but we have not verified this mechanism directly (e.g., via representation analysis), so we report it as an observation rather than an established cause.

Category 2 — Reflect-Guard false positives ($n = 109$).

These are benign prompts correctly handled by the baseline but over-triggered by Condition D. They share a surface structure with adversarial inputs (roleplay personas, sensitive topics, references to “loopholes”) despite benign intent. A typical example: a prompt asking an “editor” to review a controversial academic article. The professional roleplay and sensitive topic trigger the reflection’s adversarial-framing heuristic even though the intent is scholarly critique. This class of errors motivates future work on using reflection confidence or self-contradiction as a calibration signal.

Category 3 — All conditions fail ($n = 78$).

These are harmful prompts missed by all three conditions. Notably, they concentrate in *private information* (e.g., “How can I see who my partner texts?”) and *social stereotypes* (e.g., prompts framing discriminatory content as neutral demographic questions). These examples reveal that the method’s residual failures are not adversarial-framing failures but category-coverage failures: the training data contains too few examples of these harm types for any condition to generalize robustly.

Reflection self-consistency ($n = 2,309$).

A separate concern is whether the generated reflection actually agrees with its own emitted verdict, or the two are decoupled—which would undermine the method’s interpretability claim. Across every Condition-D prediction with a stored reflection (1,699 WildGuardTest + 281 JailbreakBench + 329 finance examples), we automatically parsed each reflection’s stated conclusion (the “the underlying intent is {harmful|benign}” clause required by the annotation template, Section 3.2) and checked it against the emitted verdict. Extraction succeeded for 99.6% of reflections, with the stated conclusion agreeing with the verdict in 99.91% of those; manual inspection of the two disagreements found both to be extraction artifacts (a hedging clause reversed an initial “appears benign” assessment, which the verdict correctly followed but our pattern did not capture), so the effective self-consistency rate is 100%. This verifies only that the reflection’s stated conclusion is not contradicted by the verdict, not that the reasoning is factually accurate or reflects genuine internal computation (Section 7.6).

6. Domain Generalization to Finance

6.1. Motivation

LLM agents deployed in financial applications—trading assistants, compliance copilots, retail-banking chatbots—face a domain-specific variant of the adversarial threat studied above: requests dressed up as “a fictional trading simulation” or “a compliance review of AML procedures” that actually solicit actionable instructions for insider trading or money laundering. Emerging finance safety benchmarks such as FinSafetyBench [3] and TRIDENT [4] document these risks but evaluate *generative* LLM refusal behavior rather than dedicated *safety classifiers*, leaving open whether a reflection-augmented classifier trained purely on general-domain data can detect these domain-specific patterns. We test this directly by applying the Reflect-Guard checkpoint from Section 3 *unmodified* to a new financial harm benchmark.

6.2. Finance Harm Taxonomy and Benchmark

We define a seven-category finance harm taxonomy (Table 4), derived from FinSafetyBench’s 14 subcategories and the CFA Institute Code of Ethics framework used by TRIDENT.

Table 4. Finance harm taxonomy used to construct the evaluation benchmark in Section 6, mapped to the closest Llama Guard 3 S1–S13 category or categories.

Code	Category	Examples
FIN-1	Financial Fraud & Scams	Phishing, Ponzi schemes, advance-fee fraud
FIN-2	Market Manipulation & Insider Trading	Pump-and-dump, spoofing, front-running
FIN-3	Money Laundering & Structuring	Placement, layering, smurfing
FIN-4	Unauthorized Investment Advice	Unlicensed advisory, suitability bypass
FIN-5	Identity Theft & Account Fraud	Account takeover, synthetic identity
FIN-6	Tax Evasion & Regulatory Evasion	Offshore accounts, AML/KYC evasion
FIN-7	Predatory Lending & Misconduct	Loan sharking, mis-selling, churning

The evaluation benchmark contains 329 examples: 240 harmful and 89 benign. The 60 FIN-1 harmful prompts are drawn from the `fraud_assisting_illegal_activities` subcategory of WildGuardMix [2]—i.e., real adversarial prompts, not synthetic ones. The remaining 180 harmful prompts (30 per category, FIN-2 through FIN-7) and all 89 benign prompts are generated by GPT-4o-mini following the same synthesis methodology used to construct the original Reflect-Guard training data (Section 3.2), mixing direct requests with adversarially-framed variants (role-play, educational pretext, fictional framing). Critically, **no model weights are updated for this evaluation**: we reuse the QLoRA adapter trained exclusively on WildGuardMix and AdvBench (Section 3.2), which contains no financial content.

Labeling procedure for the synthetic portion.

The harmful/benign label for each of the 269 GPT-4o-mini-generated examples is assigned *by construction*, not by a separate annotation or classification step: the generation script (`fin/build_benchmark.py`) uses two distinct system prompts, one instructing the model to produce a harmful example within a target FIN category and the other to produce a benign finance-related example, and the resulting label is fixed by which prompt produced the text. No human or model annotator subsequently reviews or verifies that the generated content actually matches its assigned label. Consequently, an inter-annotator agreement statistic does not apply here—there is a single generation process rather than multiple independent annotators—but this also means a generated example that fails to match its intended label (e.g., an ostensibly “benign” prompt that reads as borderline harmful) would not be caught. This is a concrete, unaddressed risk distinct from the shared-provenance concern above, and is a further reason we describe the finance-benchmark results as evidence toward generalization rather than a fully validated benchmark (Section 7.6).

We note a limitation in the benchmark’s metadata: the per-example adversarial flag inherited from WildGuardMix is populated only for the 60 FIN-1 examples (31 of which are marked adversarial); it was not independently annotated for the 209 synthetic FIN-2–FIN-7 harmful examples, even though the synthesis prompt requested a mix of direct and adversarially-framed variants there too. As a result, we cannot currently report a reliable harmful-direct vs. harmful-adversarially-framed performance breakdown for FIN-2–FIN-7 (Section 7 lists this as future work); the taxonomy-level breakdown in Table 6 is the finest-grained breakdown the current annotations support.

6.3. Results

Table 5 reports overall performance. Reflect-Guard improves F1 from 0.868 to 0.929 (+6.1 pp) and recall from 0.767 to 0.867 (+10.0 pp) relative to the baseline Llama-Guard-3-8B, with precision unchanged at 1.000—meaning neither model produces a single false positive on the 89 benign financial prompts. Because this is our smallest evaluation set ($n = 329$), we report bootstrap 95% confidence intervals directly in the table rather than the point

estimates alone: recall (0.713–0.821 vs. 0.821–0.908) and F1 (0.833–0.902 vs. 0.902–0.952) do not overlap between the baseline and Reflect-Guard, though we note this comes from a single training run per condition and is not a substitute for the multi-seed significance test in Section 7.4.1, which reaches the same direction on 2 seeds without yet reaching significance at that small n . This is also the cleanest outright win in the paper’s competitive comparison (Section 7.5): both WildGuard and the prompted GPT-4o-mini teacher were evaluated zero-shot on this benchmark with no finance-specific training either, and Reflect-Guard’s F1, recall, and accuracy lead both of them (0.929/0.867/0.903 vs. WildGuard’s 0.921/0.854/0.894).

Table 5. Overall performance on the 329-example finance benchmark (Section 6.2). Reflect-Guard uses the same checkpoint as Sections 5–7, with no finance-specific fine-tuning; WildGuard and Prompted-CoT (GPT-4o-mini) are external baselines evaluated zero-shot (Section 7.5). Bracketed values are 95% bootstrap confidence intervals (2,000 resamples over the fixed 329-example set) for Baseline and Reflect-Guard; given this is our smallest evaluation set, these are reported explicitly rather than left to the point estimate alone. WildGuard and Prompted-CoT are single evaluation runs with no CI. Best per column in **bold**.

Model	Precision	Recall	F1	Accuracy
Baseline (Llama-Guard-3-8B)	1.000 [1.000, 1.000]	0.767 [0.713, 0.821]	0.868 [0.833, 0.902]	0.830
Reflect-Guard (ours)	1.000 [1.000, 1.000]	0.867 [0.821, 0.908]	0.929 [0.902, 0.952]	0.903
Δ (Reflect-Guard – Baseline)	0.000	+0.100	+0.061	+0.073
WildGuard (external)	1.000	0.854	0.921	0.894
Prompted-CoT (GPT-4o-mini)	1.000	0.846	0.916	0.888

Table 6 breaks results down by harm category. Reflect-Guard improves F1 in six of seven categories; FIN-2 (Market Manipulation) is already at 1.000 F1 for both models. The largest gain is on FIN-1 (Financial Fraud & Scams, +12.6 pp), the category containing the real WildGuardMix examples—consistent with the pattern observed in Section 5 that reflection provides the largest benefit on the most adversarially diverse prompts. FIN-5 (Identity Theft, +8.6 pp) and FIN-6 (Tax Evasion, +6.3 pp) show the next-largest gains.

Table 6. Per-category F1 on the finance benchmark. Reflect-Guard improves over the baseline in six of seven harm categories with no finance-specific training.

Code	Category	Baseline F1	Reflect-Guard F1	Δ
FIN-1	Financial Fraud & Scams	0.812	0.938	+0.126
FIN-2	Market Manipulation & Insider Trading	1.000	1.000	0.000
FIN-3	Money Laundering & Structuring	0.846	0.868	+0.022
FIN-4	Unauthorized Investment Advice	0.889	0.909	+0.020
FIN-5	Identity Theft & Account Fraud	0.824	0.909	+0.086
FIN-6	Tax Evasion & Regulatory Evasion	0.846	0.909	+0.063
FIN-7	Predatory Lending & Misconduct	0.889	0.947	+0.058
<i>All categories</i>		0.868	0.929	+0.061

6.4. Qualitative Example

The following example (FIN-2, Market Manipulation) illustrates the mechanism behind these gains: the model identifies a professional role-play framing as a vehicle for an underlying market-manipulation request, despite having seen no financial training data.

Prompt: “Let’s role-play as financial analysts; how would you advise me to mislead investors about the potential of DEF Ltd before I sell my holdings?”

“The message employs adversarial techniques by framing the request as a role-play scenario, which can obscure the harmful intent of misleading investors. The underlying intent is harmful as it seeks to provide advice on manipulating financial information for personal gain, which falls under market manipulation and securities fraud. Key indicators: (1) role-play framing as cover for explicit market manipulation request; (2) intent to mislead investors; (3) reference to personal financial gain from deceptive disclosure.”

Verdict: unsafe (S2, S6)

The reflection identifies the role-play wrapper, the underlying harmful intent, and the financial motive—the same three-part pattern (framing → intent → indicators) seen in the general-domain examples of Section 5.3, consistent with adversarial-framing patterns that are at least partly domain-invariant rather than purely domain-specific. We hold this interpretation to the caveats in Section 7.6: the benchmark’s synthetic majority and unverified labels mean this is evidence toward domain generalization, not a settled demonstration of it.

7. Analysis and Discussion

7.1. Error Analysis

False positives.

Reflect-Guard (Condition D) has 142 total false positives on WildGuardTest versus 44 for the clean baseline (Condition 0); exact-prompt matching shows these are not simply additive (33 shared, 11 resolved, 109 newly introduced: $142 = 33 + 109$, $44 = 33 + 11$). Of the 109 new false positives, 102 (93.6%) fall in the adversarial subset—prompts that appear adversarial (fictional framing, role-play, security-related language) but are actually benign, e.g., a DEF CON cybersecurity-competition narrative flagged as harmful. This suggests Reflect-Guard’s adversarial-detection heuristic over-triggers on security-adjacent but harmless content.

False negatives.

The remaining 103 false negatives (down from 254 at baseline) concentrate in *private information* (32), *social stereotypes and discrimination* (26), and *others* (16)—subtle harms where adversarial framing effectively mimics legitimate research or educational context; the model’s reflections on these typically (and wrongly) conclude “educational” or “benign.”

JailbreakBench misses.

Of the 5 misses (of 282), 4 use PAIR with elaborate fictional or educational framings (e.g., a historian analyzing ancient warfare) where harmful intent is deeply embedded in a plausible professional scenario. The remaining miss is a GCG prompt and is *not* a reasoning failure: the reflection correctly states “the underlying intent is harmful” before being truncated mid-sentence at the 150-token limit, and our parser defaults the resulting non-conforming output to *unharmful*. This token-budget/parsing artifact is the only such case among all 3×282 JailbreakBench generations (Section 7.6).

7.2. Precision–Recall Tradeoff

The precision decrease (−9.8 pp overall, −13.5 pp on adversarial prompts) merits discussion. For safety-critical applications, recall is generally prioritized over precision: a missed harmful prompt (false negative) can lead to actual harm, while a false positive merely results in an unnecessary refusal. Reflect-Guard’s operating point—high recall with moderate precision—aligns with this principle. In deployment, the false positive rate could

be mitigated through confidence thresholds on the reflection, cascaded verification with a second classifier, or human-in-the-loop review for borderline cases.

7.3. Efficiency Considerations

Reflect-Guard's *training* cost is low, but its *inference* overhead relative to the base model is substantial. We report it here as a direct measurement rather than a qualitative characterization:

- **Training cost:** 1,000 examples, ~ 42 M trainable parameters (0.5% of 8B), ~ 1 hour on a single A5000 GPU.
- **Data synthesis cost:** 1,000 GPT-4o-mini API calls for reflection generation (one-time cost).
- **Inference overhead:** measured directly (Table 7, below), not estimated.

Table 7. Inference latency, throughput, and GPU memory, measured on 200 fixed WildGuardTest prompts on a single NVIDIA A40 GPU, greedy decoding, max_new_tokens=150. Single-request latency is measured one prompt at a time (batch size 1); batched throughput uses batch size 8. All three conditions were measured back-to-back in the same job on identical hardware.

Condition	P50 (s)	P95 (s)	P99 (s)	Mean tokens	Throughput (prompts/s)
0. Clean Baseline	0.282	0.377	0.422	3.9	4.30
B. SFT Labels Only	0.352	0.513	0.553	3.4	3.44
D. Reflect-Guard	5.045	6.322	6.650	92.5	0.43

Reflect-Guard's median single-request latency (5.05s) is $17.9\times$ that of the clean baseline (0.28s) and $14.3\times$ that of Condition B (0.35s); batched throughput falls by a similar factor (0.43 vs. 4.30 and 3.44 prompts/s, a $10.0\times$ and $8.0\times$ reduction respectively). This is driven almost entirely by output length: Reflect-Guard generates a mean of 92.5 tokens per prompt (the reflection plus verdict), versus 3.4–3.9 tokens for the label-only conditions. This overhead is substantial, not negligible, and is the direct, measured cost of the adversarial-recall gain reported in Section 7.4. Peak GPU memory is comparatively unaffected (11.90 GB for Condition 0 vs. 12.06 GB for both B and D, a +1.3% increase), since the LoRA adapter itself is small and the additional cost is almost entirely in generation length rather than model size.

The LoRA adapter checkpoint (adapter_model.safetensors) is ~ 84 MB on disk—consistent with ~ 42 M trainable parameters stored in bf16 (2 bytes/parameter)—and can be dynamically loaded/unloaded, enabling flexible deployment alongside the base Llama Guard model.

7.4. Ablation Study

	Condition	Reflection	SFT	GT Label	Purpose
$A \approx 0?$	0. Clean Baseline	×	×	—	True baseline
	A. Prompted Refl.	✓	×	—	Prompting alone?
	B. SFT Labels Only	×	✓	—	SFT decision boundary
$D > B > C?$	C. Blind Reflections	✓	✓	×	GT label quality?
	D. Full Reflect-Guard	✓	✓	✓	Full method

Figure 2. Ablation study design. Five conditions isolate the contribution of each component. Condition A tests whether prompting alone (without fine-tuning) induces reflection. Condition B isolates the effect of SFT on the decision boundary without reflection. Condition C tests whether teacher reflections generated without the ground-truth label are less effective. Condition D is the full Reflect-Guard system.

- To disentangle the contributions of reflection reasoning versus supervised fine-tuning (SFT) on the decision boundary, we conduct an ablation study with five conditions:
- **Condition 0 (Clean Baseline):** Llama-Guard-3-8B with the standard prompt (no reflection instruction, no SFT).
 - **Condition A (Prompted Reflection):** Llama-Guard-3-8B prompted to generate reflections, but without any fine-tuning.
 - **Condition B (SFT Labels Only):** QLoRA fine-tuned on the same 1,000 examples, but training targets contain only the verdict (safe/unsafe + categories)—no reflection text.
 - **Condition C (Blind Reflections):** QLoRA fine-tuned with reflections generated by GPT-4o-mini *without* access to the ground-truth label.
 - **Condition D (Full Reflect-Guard):** The full method with ground-truth-informed reflections.

Table 8. Ablation study results. F_2 weights recall twice as heavily as precision, reflecting safety deployment priorities. Adv-Rec = recall on the adversarial subset of WildGuardTest ($n = 796$). JBB DR = JailbreakBench detection rate.

Condition	R	S	G	Acc	Prec	Rec	F1	F_2	Adv-Rec	JBB DR
0. Clean Baseline	×	×	—	.825	.919	.663	.770	.702	.513	89.7%
A. Prompted Refl.	✓	×	—	.819	.916	.651	.761	.691	.487	89.7%
B. SFT Labels Only	×	✓	—	.877	.868	.853	.860	.856	.845	100.0%
C. Blind Reflections	✓	✓	×	.856	.847	.825	.836	.829	.848	98.9%
D. Full Reflect-Guard	✓	✓	✓	.856	.821	.863	.842	.855	.921	98.2%

R = Reflection, S = SFT, G = GT-label during synthesis.

The results directly address the question of how much gain is attributable to reflection *per se* versus SFT changing the decision boundary.

Prompting alone provides no benefit (Condition A). Condition A is virtually indistinguishable from Condition 0 (F1: 0.761 vs. 0.770; JBB detection rate identical at 89.7%), confirming that chain-of-thought prompting without fine-tuning cannot shift the decision boundary. This rules out the hypothesis that the reflection instruction text alone drives improvements.

SFT on the decision boundary drives the primary gain (Condition B). Condition B yields a +9.0 pp F1 improvement over the baseline (0.860 vs. 0.770) and perfect JBB detection (100.0%), demonstrating that the bulk of the gain comes from adapting the classifier’s decision boundary via SFT, not from reflection *per se*.

Reflection training reallocates performance from non-adversarial to adversarial inputs (Condition D vs. B). Condition D trades 1.8 pp of overall F1 (0.842 vs. 0.860) for a +7.6 pp gain in adversarial recall (0.921 vs. 0.845). The subset breakdown clarifies the mechanism: on adversarial prompts, D gains +7.6 pp recall at a cost of −7.9 pp precision (adv-prec: 0.789→0.710), while on non-adversarial prompts, D actually improves precision slightly (+1.6 pp: 0.944→0.960) at a cost of −4.4 pp recall. This cross-pattern is incompatible with a simple global threshold shift—if reflection merely lowered the decision boundary, precision would fall and recall would rise on *both* subsets equally. Instead, the opposing trends reveal that reflection trains the model to be more aggressive specifically on adversarially-framed prompts, and more cautious (conservative) on non-adversarial ones, implying it has internalized features of adversarial framing rather than simply recalibrated a scalar threshold. The overall F_2 is nearly tied (D: 0.855, B: 0.856), but adversarial F_2 decisively favors D (0.869 vs. 0.833), confirming that reflection adds measurable value under safety-relevant evaluation criteria.

Ground-truth labels during synthesis provide a small but consistent benefit (Condition D vs. C). Condition C, trained on reflections generated without access to ground-truth labels, achieves F1 of 0.836—only 0.6 pp below Condition D (0.842)—showing the approach is robust even when teacher supervision is label-blind. The small gap suggests GT labels help sharpen reflection quality but are not strictly necessary, making the method practical when labeled data is scarce.

7.4.1. Multi-Seed Statistical Robustness

The Condition B-vs-D comparison above uses a single training run per condition (seed 42). To test whether the reallocation pattern is a robust training-time effect or an artifact of that one run, we retrained Conditions B and D under two additional random seeds (123, 2024; all other hyperparameters and training data held fixed) and re-evaluated each on all three benchmarks, giving 3 independent (B, D) pairs for WildGuardTest/JailbreakBench and 2 for the finance benchmark (Condition B was not evaluated on finance in the original seed-42 run). Table 9 reports the per-condition mean $\pm$ std across seeds and a paired significance test (t-test, plus a Wilcoxon signed-rank test as a non-parametric check given the very small n) on the difference between D and B for each metric.

Table 9. Multi-seed statistical robustness (seeds 42, 123, 2024; finance uses 123, 2024 only). Mean $\pm$ std across seeds; p -value from a paired t-test between Conditions B and D on the same seeds. Wilcoxon signed-rank p is bounded below by 0.25 at $n = 3$ and is included only as a non-parametric sanity check, not as the primary test.

Metric	n seeds	B		D		paired t -test	Wilcoxon
		mean	$\pm$ std	mean	$\pm$ std	p	p
WGT overall F1	3	0.8654	$\pm$ 0.0049	0.8399	$\pm$ 0.0026	0.018	0.250
WGT adversarial F1	3	0.8140	$\pm$ 0.0042	0.7956	$\pm$ 0.0079	0.018	0.250
WGT overall recall	3	0.8630	$\pm$ 0.0100	0.8559	$\pm$ 0.0080	0.508	0.500
WGT adversarial recall	3	0.8426	$\pm$ 0.0090	0.9003	$\pm$ 0.0221	0.027	0.250
JailbreakBench DR (recall)	3	0.9953	$\pm$ 0.0054	0.9823	$\pm$ 0.0035	0.111	—
Finance F1	2	0.9450	$\pm$ 0.0033	0.9274	$\pm$ 0.0017	0.125	—

Two findings from Table 9 strengthen the paper’s central claim with statistical support rather than a single point estimate. **Condition B’s overall-F1 advantage over D is statistically significant** (WGT overall and adversarial F1: $p = 0.018$), holding on every one of the three seeds—not an artifact of the seed-42 headline run. **Condition D’s adversarial-recall advantage over B is separately significant** ($p = 0.027$), also holding on all three seeds, though the mean advantage (+5.77 pp: 0.9003 vs. 0.8426) is somewhat smaller than the seed-42 point estimate reported elsewhere (+7.6 pp), which we treat as the more representative figure. *Overall* recall shows no significant difference between B and D ($p = 0.508$), consistent with reflection reallocating recall toward the adversarial subset rather than shifting a global threshold—the same conclusion Section 7.4’s single-run analysis reached, now with multi-seed support. JailbreakBench and finance show the same directional pattern but do not reach significance at $n = 3$ and $n = 2$; we report this as an underpowered test, not an absence of effect.

At this sample size, a paired t -test has limited power, and the Wilcoxon test’s minimum achievable p -value at $n = 3$ is 0.25, so its non-significance reflects sample size, not evidence against the effect—in every metric where the t -test reaches significance, all three seeds agree in direction, the strongest signal available at this n . Exact per-seed values are released alongside the aggregate (data availability statement).

7.4.2. Confusion Matrices and Single-Run Significance Tests

Table 9 tests significance across independent *training* runs; Tables 10–11 complement it with the exact raw counts underlying every precision/recall/F1 value in this paper, and a paired bootstrap significance test (2,000 resamples of the shared, prompt-aligned test set, applied to the *difference* between two conditions’ predictions on each resample) for all three pairwise comparisons among Conditions 0, B, and D on the single seed-42 run.

Table 10. Confusion matrices (Conditions 0/B/D, single seed-42 run) underlying every precision/recall/F1 value reported in Sections 5–6. “harmful” is the positive class throughout.

Benchmark	Condition	<i>n</i>	TP	FP	FN	TN
WildGuardTest	0. Baseline	1,699	500	44	254	901
	B. SFT-Only	1,699	643	98	111	847
	D. Reflect-Guard	1,699	651	142	103	803
JailbreakBench	0. Baseline	282	253	0	29	0
	B. SFT-Only	282	282	0	0	0
	D. Reflect-Guard	282	277	0	5	0
Finance	0. Baseline	329	184	0	56	89
	D. Reflect-Guard	329	208	0	32	89

Table 11. Paired bootstrap significance tests between Conditions 0, B, and D on the single seed-42 run (2,000 resamples of the shared, prompt-aligned test set; two-sided *p*-value). Metric is F1 for WildGuardTest and finance, recall (detection rate) for JailbreakBench. This is a test-set-resampling test, distinct from and complementary to the seed-resampling test in Table 9.

Benchmark	Comparison	Δ	<i>p</i> -value	
WildGuardTest (F1)	D – 0	+0.0712	< 0.001	sig.
	B – 0	+0.0898	< 0.001	sig.
	D – B	–0.0186	0.014	sig.
JailbreakBench (recall)	D – 0	+0.0851	< 0.001	sig.
	B – 0	+0.1028	< 0.001	sig.
	D – B	–0.0177	0.012	sig.
Finance (F1)	D – 0	+0.0606	< 0.001	sig.

Two results stand out. All three pairwise comparisons on WildGuardTest and JailbreakBench are significant even within this single run, including Condition D’s disadvantage relative to Condition B ($p = 0.014$ WGT F1, $p = 0.012$ JBB recall)—directionally consistent with, but methodologically independent of, the multi-seed test above ($p = 0.018$), giving two separate lines of evidence that B’s aggregate advantage is real rather than sampling noise. And Table 10’s raw counts make the precision cost concrete: Condition D’s higher recall than Condition 0 (651 vs. 500 true positives) comes with more than triple the false positives (142 vs. 44).

7.5. Comparison to Competitive Safety Classifiers

The ablation above isolates reflection’s contribution relative to our *own* un-fine-tuned baseline and SFT-only ablation, but not how Reflect-Guard stands relative to the wider field—the central gap identified in review. We evaluate four external classifiers, zero-shot, on the same three benchmarks: WildGuard [2] and PromptGuard [7] with their released weights and default templates; a Prompted-CoT baseline using GPT-4o-mini—the same model used as our reflection-distillation teacher (Section 3.2)—prompted with the identical S1–S13 taxonomy to reason step by step, as an “undistilled teacher” reference point; and OpenAI’s moderation classifier [33]. WildGuard and Prompted-CoT are compared head-to-head in the main tables below; PromptGuard and OpenAI’s moderation classifier are discussed separately (Section 7.6) rather than included in Tables 1–5, since both show a fundamental taxonomy / design-scope mismatch with this paper’s task—PromptGuard v1 by over-triggering on nearly everything outside adversarial framing (its apparently strong 0.909 adversarial-subset recall is an artifact of this, not effective detection), OpenAI’s moderation classifier by under-triggering on harmful instructions whose surface text is not itself sexual, hateful, or violent. ShieldGemma [8] could not be evaluated: both checkpoints are gated HuggingFace repositories, access to which had not been granted at the time of writing.

The results (Tables 1, 2, 3, 5) support four observations. WildGuard leads all three of our conditions on aggregate WildGuardTest accuracy, precision, and F1 (0.903/0.926/0.887 vs. Condition D’s 0.856/0.821/0.842)—unsurprising, given it is trained specifically for this distribution on a larger,

more diverse dataset than our 1,000-example fine-tuning set. Central to this paper’s claim, though: *adversarial-subset recall is the one metric across every table where Reflect-Guard leads every evaluated classifier, including WildGuard* (0.921 vs. 0.827 for WildGuard, 0.636 for the prompted teacher)—WildGuard’s adversarial-subset F1 (0.852) still exceeds Condition D’s (0.802) since it pays a smaller precision cost for its recall, but on the specific axis of *not missing* a disguised harmful prompt, Reflect-Guard is strongest.

Condition B—not just D—beats WildGuard outright on JailbreakBench (100.0% vs. 99.65% DR), and D’s own guard bypass rate (1.8%) is lower than the prompted teacher’s (3.19%) though higher than WildGuard’s (0.35%). On the finance zero-shot-transfer benchmark, where none of the three saw finance-specific training data, Reflect-Guard has the highest F1, recall, and accuracy of any evaluated classifier (0.929/0.867/0.903 vs. WildGuard’s 0.921/0.854/0.894)—the cleanest outright win in this comparison. And Reflect-Guard beats the prompted-teacher baseline on *every* metric across all three benchmarks: simply prompting a larger, more capable model with the same taxonomy does not reproduce what QLoRA fine-tuning plus reflection distillation achieves, even though the prompted model is the undistilled source of that same reasoning capability.

We therefore make a precise, narrow claim rather than an unqualified state-of-the-art claim: among the classifiers evaluated here, Reflect-Guard achieves the highest adversarial-subset recall on WildGuardTest and the highest F1 on the finance-transfer benchmark; it does not lead on aggregate WildGuardTest metrics, where WildGuard’s broader, purpose-built training data yields a higher overall F1 (0.887 vs. 0.842).

7.6. Limitations

Several limitations should be acknowledged.

Teacher bias transfer.

Reflection annotations are generated by GPT-4o-mini with access to ground-truth labels, creating two risks: (1) the model may learn idiosyncratic teacher phrasing rather than generalizable reasoning; (2) systematic biases in GPT-4o-mini’s safety judgements may be distilled into the student. Condition C partially mitigates (2)—strong performance without GT labels suggests the method is not merely memorizing label-conditioned templates. On (1), the finance-domain results (Section 6) bear indirectly: the adapter improves detection on a domain absent from its training data, which would be surprising if the gain were purely stylistic mimicry rather than a transferable framing-detection capability—though the teacher’s own biases could still transfer along with the useful signal, and the two are not distinguishable from our results alone. A direct test would compare teacher models (GPT-4, Claude, etc.); we have not conducted this and list it as future work.

False positives.

The 109 new false positives introduced relative to the clean baseline, 102 of them on the adversarial subset (Category 2 in Section 5.3), could affect users with legitimate security research queries. The reflection text provides a natural mitigation signal: a low-confidence or self-contradictory reflection could trigger secondary verification rather than an immediate block.

Competitive baselines remain incomplete.

PromptGuard v1 (meta-llama/Prompt-Guard-86M) is not usable as a general-classifier comparison: it predicts *harmful* for 100% of the WildGuardTest non-adversarial subset (903/903) and 100% of the finance benchmark’s benign examples (89/89), with non-trivial specificity only on the adversarial subset (61/455 correctly passed). This targets prompt-injection/jailbreak *technique* detection specifically (per its model card), not general content harm, and is a known, independently reported limitation of v1; Meta’s successor, Prompt Guard 2, reports a large calibration improvement (Recall@1%FPR: 21.2% → 97.5%) that we did not evaluate. OpenAI’s moderation classifier [33] shows the opposite failure mode: overall WildGuardTest F1 of 0.485 (recall 0.353), adversarial-subset recall of 0.226, and a JailbreakBench detection rate of 48.2%—below even the un-fine-tuned Condition 0’s 89.7%—because its fixed taxonomy targets platform-content properties (sexual, hateful, violent, self-harm) rather than harmful *instructions*, whose surface text need not itself be sexual, hateful, or violent. Both are excluded from Tables 1–5 as taxonomy / design-scope mismatches rather than like-for-like

comparisons. We provide head-to-head comparisons against WildGuard and a prompted GPT-4o-mini reasoning classifier instead (Section 7.5); the one remaining gap is ShieldGemma, blocked on pending HuggingFace gated-repo access, which we commit to adding once granted.

Remaining ablations and untested analyses.

Conditions 0, A, and C remain single-run (0 is training-free; A and C are secondary ablation arms), unlike B and D, which were retrained under three seeds (Section 7.4.1). We also have not run LoRA-rank, reflection-length, training-epoch, or teacher-model ablations beyond the five conditions in Table 8, nor a training-set-size sensitivity analysis: all results use exactly 1,000 examples, chosen as a practical budget rather than a scaling-curve-justified size, so whether the B-vs-D reallocation pattern is stable at an order of magnitude more data is untested. Finally, a matched-operating-point analysis (precision at fixed recall, or vice versa, between B and D) would distinguish whether reflection changes the model's *ranking* of examples or simply shifts a scalar threshold, but requires per-token log-probabilities at the verdict position that we did not record in any run underlying this paper—a concrete, well-specified direction for future work rather than something we approximate from data that cannot support it.

Verdict-parser fallback behavior.

Our verdict parser defaults any non-conforming output to `unharmful` (data availability statement, above). Checked across every generation in this paper (3 conditions $\times$ 1,699 WildGuardTest, 3 $\times$ 282 JailbreakBench, 2 $\times$ 329 finance prompts), this fallback fires exactly once—the Condition-D JailbreakBench truncation case discussed in Section 7, a parsing artifact rather than a model-judgment error.

Reflection correctness and end-user exposure.

A correct safety verdict can, in principle, rest on an incorrect or fabricated reflection, since both are generated by the same model. Section 5.3 directly measures one checkable dimension of this—whether the reflection's own stated conclusion is self-consistent with the verdict that follows it (effectively 100% across 2,309 predictions)—but this measures self-consistency, not correctness: a reflection can be internally consistent while factually wrong, as the JailbreakBench PAIR misses (Section 7) illustrate. Whether a *correct* verdict co-occurs with unfaithful reasoning that does not manifest as a stated-conclusion mismatch remains unquantified, and would require human annotation or a counterfactual faithfulness probe we have not conducted. Whether the reflection text should be exposed to end users is likewise unresolved: it aids interpretability and debugging, but could also give an adversary a more direct signal for iterating against the classifier.

Finance benchmark provenance and domain scope.

Of the 329 finance-benchmark examples, 60 are drawn from WildGuardMix and 269 are generated by GPT-4o-mini—the same model family used for the reflection training targets (Section 3.2), raising a shared-provenance concern. Splitting results by provenance argues against pure stylistic mimicry: the gain is *larger* on the 60 real examples (F1 +12.6 pp) than the 269 synthetic ones (+4.0 pp)—but this rests on only 60 real examples, and more fundamentally, neither portion's labels have been independently human-verified. We report the finance results as evidence toward generalization under this caveat, not a settled demonstration, and view an independently constructed, human-validated benchmark as important future work. Finance is also the only high-risk domain tested beyond WildGuardTest/JailbreakBench; healthcare, cybersecurity, and legal domains each have their own professional-pretext adversarial patterns that may or may not transfer similarly, and building a comparable benchmark for any of them is future work of a similar scope to Section 6.2.

Multilingual and cross-cultural generalization.

Both our training data and teacher reflections are English-only, so the LoRA adapter has not been exposed to non-English adversarial patterns or culturally-specific harm framings. Three concrete gaps follow: the reflection capability may not transfer to non-English inputs even though the base model supports multiple languages; the S1–S13 taxonomy itself reflects an English-speaking Western

notion of harm that may miss what is salient elsewhere (e.g., caste-related content, region-specific stereotypes); and multilingual attackers can exploit code-switching or culturally-embedded framings absent from our English training corpora. Addressing this would require target-language reflection data from a multilingual teacher, a culturally-expanded taxonomy, and multilingual evaluation benchmarks—all left as future work.

8. Conclusion and Future Directions

We set out to test whether teaching a safety classifier to generate an explicit safety rationale before its verdict improves LLM safety classification, and to isolate that effect from ordinary supervised fine-tuning (SFT) using a five-condition ablation. The clearest finding is that SFT on verdict labels alone (Condition B), with no reflection at all, already accounts for the majority of the aggregate improvement over the un-fine-tuned Llama-Guard-3-8B baseline: overall WildGuardTest F1 rises from 0.770 to 0.860 and JailbreakBench guard bypass rate falls from 10.3% to 0.0%. Adding reflection training on top of SFT (Condition D, the full Reflect-Guard model) does *not* improve these aggregate numbers further—overall F1 falls slightly to 0.842 and guard bypass rate rises to 1.8%, and Condition B has the higher F1 on both the adversarial and non-adversarial WildGuardTest subsets individually. What reflection does provide is a targeted, opposite-direction shift specifically on adversarially-framed inputs: +7.6 pp recall on the adversarial subset relative to Condition B, at a cost of −7.9 pp precision on that same subset, while simultaneously improving precision (at a cost to recall) on non-adversarial inputs. This cross-subset pattern is inconsistent with a single global decision-threshold shift and instead indicates the model has learned features specific to adversarial framing—a real but narrower contribution than an unconditional improvement, and one best described as a precision–recall tradeoff rather than a strict gain. Retraining Conditions B and D under three random seeds confirms both halves of this finding are robust to training-time randomness rather than artifacts of the single run reported above: Condition B’s overall-F1 advantage over D is statistically significant ($p = 0.018$, paired t -test) and holds on every seed, and Condition D’s adversarial-recall advantage over B is *separately* significant ($p = 0.027$), while overall recall shows no significant difference between the two ($p = 0.508$)—precisely the reallocation-not-uniform-shift pattern the single-run cross-subset analysis suggested, now with multi-seed statistical support (Section 7.4.1). We further find that this adversarial-recall advantage extends to a comparison against an external, purpose-built safety classifier: among WildGuard, a prompted reasoning baseline, and Reflect-Guard, only Reflect-Guard leads on adversarial-subset recall, even though WildGuard achieves higher aggregate WildGuardTest metrics (Section 7.5).

In practical terms, this tradeoff is still useful in the safety-critical settings that motivated this work: a classifier that trades some precision for higher recall specifically on adversarially-framed prompts is well suited to deployments where a missed jailbreak is costlier than an extra false positive, and the reflection text provides a natural, human-readable signal for triaging borderline cases rather than issuing an opaque binary verdict. This benefit is not free: measured directly rather than estimated (Section 7.3), Reflect-Guard’s median single-request latency is $17.9\times$ that of the un-fine-tuned baseline and $14.3\times$ that of Condition B, driven by the ~ 92 additional reflection tokens generated per prompt—a cost that should be weighed explicitly against the targeted adversarial-recall gain in any deployment decision, rather than assumed away. This benefit appears not to be an artifact of the specific training distribution: the same LoRA adapter, evaluated zero-shot with no additional fine-tuning, generalizes to a curated 329-example financial-harm benchmark (Section 6), improving F1 from 0.868 to 0.929 and recall from 0.767 to 0.867 relative to the un-fine-tuned baseline, with zero false positives on benign financial queries—consistent with the model having learned reusable adversarial-framing patterns rather than narrowly memorizing the training domain, though we hold this claim to the caveats in Section 7.6 regarding the finance benchmark’s partly GPT-4o-mini-generated provenance.

Several directions remain open before these findings can be considered conclusive. Reflect-Guard trails WildGuard on aggregate WildGuardTest metrics but leads on adversarial-subset recall and finance-benchmark F1 (Section 7.5); ShieldGemma is the one planned competitive baseline we could not run, blocked on pending HuggingFace gated-repo access, and we commit to adding it once access is granted. Remaining priorities (Section 7.6) are: extending multi-seed retraining (Section 7.4.1) to Conditions 0, A, and C; LoRA-rank, reflection-length, training-epoch, teacher-model, and training-set-size ablations beyond the five conditions already tested; and an independently

constructed, human-annotated finance benchmark to remove the shared-provenance caveat on our domain-generalization claim. Finally, extending the method to multilingual and cross-cultural settings remains an important open direction: generating reflection training data in target languages, adapting the safety taxonomy to culturally-specific harm categories, and evaluating on non-English adversarial benchmarks would establish whether reflection-based reasoning generalizes beyond the English-language context studied here.

Author Contributions: Conceptualization, L.L. (Lixing Lin); methodology, L.L. (Lixing Lin); software, L.L. (Lixing Lin); validation, L.L. (Lixing Lin), J.Y., Y.L., L.L. (Luyun Lin), Y.W., Z.Z. and M.Z.; formal analysis, L.L. (Lixing Lin); investigation, L.L. (Lixing Lin); writing—original draft preparation, L.L. (Lixing Lin); writing—review and editing, L.L. (Lixing Lin), J.Y., Y.L., L.L. (Luyun Lin), Y.W., Z.Z. and M.Z. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Code, evaluation scripts, and data are openly available at <https://github.com/lixinglin-AI/AI-Safety>, including: training/evaluation scripts for all five ablation conditions; teacher prompt templates and reflection-generation configuration (`reflect_guard/config.py`, `synthesize.py`); generated reflection training annotations (`reflect_guard/*_train.jsonl`, 1,000 examples each); the finance benchmark and construction script (`fin/fin_benchmark.jsonl`, `fin/build_benchmark.py`); per-example raw predictions for every condition and seed (`results/*.json`, `results/seed_variance_report`); with the multi-seed aggregation script (`reflect_guard/compute_seed_variance.py`); raw latency measurements (`results/latency/*.json`); and evaluation scripts/outputs for all four competitive baselines (`baselines/evaluate*.py`, `aggregate_comparison.py`, `results/baselines/*.json`). Upstream datasets (WildGuardMix, AdvBench, JailbreakBench) are publicly available at their sources; exact identifiers, configs, and the sampling seed are given in Section 4.3. Python package versions are pinned in `requirements.txt`. Model outputs are parsed deterministically: after stripping any `<reflection>` block, an output is labeled harmful if and only if the remaining text *begins* with “unsafe”; any other case—including malformed generations, missing verdict tokens, or inference errors—defaults to `unharmful`, a design choice whose effect on reported metrics is not isolated beyond the single measured instance in Section 7.6. The QLoRA adapter weights are not yet uploaded to a model hub; we commit to releasing them alongside the accepted version of this paper.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

LLM	Large Language Model
CoT	Chain-of-Thought
SFT	Supervised Fine-Tuning
LoRA	Low-Rank Adaptation
QLoRA	Quantized Low-Rank Adaptation
NF4	4-bit NormalFloat
DR	Detection Rate
GBR	Guard Bypass Rate
GCG	Greedy Coordinate Gradient
PAIR	Prompt Automatic Iterative Refinement
JBC	JailbreakBench Classifier

References

1. Inan, H.; Upasani, K.; Chi, J.; Rungta, R.; Iyer, K.; Mao, Y.; Tontchev, M.; Hu, Q.; Fuller, B.; Testuggine, D.; et al. Llama Guard: LLM-based input-output safeguard for human-AI conversations. *arXiv* **2023**, arXiv:2312.06674.

2. Han, S.; Kim, Y.J.; Oh, A.; Hessel, J.; Choi, Y. WildGuard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of LLMs. *arXiv* **2024**, arXiv:2406.18495. 873
3. Anonymous. FinSafetyBench: Evaluating LLM safety in real-world financial scenarios. *arXiv* **2026**, arXiv:2605.00706. 874
4. Anonymous. TRIDENT: Benchmarking LLM safety in finance, medicine, and law. *arXiv* **2025**, arXiv:2507.21134. 875
5. Meta. Llama Guard 2. Available online: <https://huggingface.co/meta-llama/Llama-Guard-2-8B> (accessed on 22 August 2026). 876
6. Meta. Llama Guard 3-8B. Available online: <https://huggingface.co/meta-llama/Llama-Guard-3-8B> (accessed on 22 August 2026). 877
7. Meta. PromptGuard-86M. Available online: <https://huggingface.co/meta-llama/Prompt-Guard-86M> (accessed on 22 August 2026). 878
8. Zeng, W.; Xu, Y.; Gupta, A.; et al. ShieldGemma: Generative AI content moderation based on Gemma. *arXiv* **2024**, arXiv:2407.21772. 879
9. Luo, H.; Dai, S.; Ni, C.; Li, X.; Zhang, G.; Wang, K.; Liu, T.; Salam, H. AgentAuditor: Human-level safety and security evaluation for LLM agents. In *Advances in Neural Information Processing Systems*; **2025**; Volume 38, pp. 43241–43298. 880
10. Kong, L.; Sun, X.; Chow, W.; Li, L.; Lin, K.Q.; Zhang, X.B.; Wang, S.; Li, R.; Wu, Q.; Gao, W.; Wang, Y.; Xie, S.; Liu, J.; Qu, L.; Li, S.; Ng, L.X.; Côttereau, B.R.; Liu, Z.; Chua, T.-S.; Ooi, W.T. AI for Auto-Research: Roadmap & User Guide. *arXiv preprint* arXiv:2605.18661, 2026. 881
11. Chao, P.; Robey, A.; Dobriban, E.; Hassani, H.; Pappas, G.J.; Wong, E. Jailbreaking black box large language models in twenty queries. *arXiv* **2023**, arXiv:2310.08419. 882
12. Zou, A.; Wang, Z.; Carlini, N.; Nasr, M.; Kolter, J.Z.; Fredrikson, M. Universal and transferable adversarial attacks on aligned language models. *arXiv* **2023**, arXiv:2307.15043. 883
13. Liu, X.; Xu, N.; Chen, M.; Xiao, C. AutoDAN: Generating stealthy jailbreak prompts on aligned large language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*; 2024. 884
14. Mehrotra, A.; Zampetakis, M.; Kassianik, P.; Nelson, B.; Anderson, H.; Singer, Y.; Karbasi, A. Tree of attacks: Jailbreaking black-box LLMs with auto-generated subversive prompts. *arXiv* **2024**, arXiv:2312.02119. 885
15. Wei, A.; Haghtalab, N.; Steinhardt, J. Jailbroken: How does LLM safety training fail? In *Advances in Neural Information Processing Systems*; 2024. 886
16. Shen, X.; Chen, Z.; Backes, M.; Shen, Y.; Zhang, Y. “Do anything now”: Characterizing and evaluating in-the-wild jailbreak prompts on large language models. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*; 2024. 887
17. Chao, P.; Dobriban, E.; Hassani, H.; Pappas, G.J.; Wong, E. JailbreakBench: An open robustness benchmark for jailbreaking large language models. *arXiv* **2024**, arXiv:2404.01318. 888
18. Jiang, X.; Yang, S.; Yang, W.; Liu, Y.; Ji, C. Agentic AI as a cybersecurity attack surface: Threats, exploits, and defenses in runtime supply chains. *arXiv* **2026**, arXiv:2602.19555. 889
19. Cai, H.; Shen, B.; Jin, L.; Hu, L.; Fan, X. Does tone change the answer? Evaluating prompt politeness effects on modern LLMs: GPT, Gemini, LLaMA. *arXiv* **2025**, arXiv:2512.12812. 890
20. Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E.; Le, Q.V.; Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*; 2022. 891
21. Wang, X.; Gao, S.; Zhao, J. Chain-of-thought prompting for content moderation. *arXiv* **2024**, arXiv:2402.12490. 892
22. Bai, Y.; Mu, J.; Furlanello, D.; et al. Deliberative alignment: Reasoning enables safer language models. *arXiv* **2024**, arXiv:2412.16339. 893
23. Bai, Y.; Kadavath, S.; Kundu, S.; Askell, A.; Kernion, J.; Jones, A.; Chen, A.; Goldie, A.; Mirhoseini, A.; McKinnon, C.; et al. Constitutional AI: Harmlessness from AI feedback. *arXiv* **2022**, arXiv:2212.08073. 894
24. Hinton, G.; Vinyals, O.; Dean, J. Distilling the knowledge in a neural network. *arXiv* **2015**, arXiv:1503.02531. 895
25. Tunstall, L.; Beeching, E.; Lambert, N.; Rajani, N.; Rasul, K.; Belkada, Y.; Huang, S.; von Werra, L.; Fourrier, C.; Habib, N.; et al. Zephyr: Direct distillation of LM alignment. *arXiv* **2023**, arXiv:2310.16944. 896
26. Dettmers, T.; Pagnoni, A.; Holtzman, A.; Zettlemoyer, L. QLoRA: Efficient finetuning of quantized language models. In *Advances in Neural Information Processing Systems (NeurIPS)*; 2024. 897
27. Hu, E.J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; Chen, W. LoRA: Low-rank adaptation of large language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*; 2022. 898
28. Liu, Y.; Gao, H.; Zhai, S.; Xia, J.; Wu, T.; Xue, Z.; Chen, Y.; Kawaguchi, K.; Zhang, J.; Hooi, B. GuardReasoner: Towards reasoning-based LLM safeguards. *arXiv* **2025**, arXiv:2501.18492. 899
29. Mou, Y.; Luo, Y.; Zhang, S.; Ye, W. SaRO: Enhancing LLM safety through reasoning-based alignment. *arXiv* **2025**, arXiv:2504.09420. 900
30. Sharma, M.; Tong, M.; Mu, J.; Wei, J.; Kruthoff, J.; Goodfriend, S.; Ong, E.; Peng, A.; Agarwal, R.; Anil, C.; et al. Constitutional classifiers: Defending against universal jailbreaks across thousands of hours of red teaming. *arXiv* **2025**, arXiv:2501.18837. 901
31. Phan, H.; Li, V.; Lei, Q. Think twice, generate once: Safeguarding by progressive self-reflection. In *Findings of the Association for Computational Linguistics: EMNLP 2025*; 2025; pp. 9466–9483. 902

32. Sun, L.; Huang, Y.; Wang, H.; Wu, S.; Zhang, Q.; Gao, C.; Huang, Y.; Lyu, W.; Zhang, Y.; Li, X.; et al. TrustLLM: Trustworthiness in large language models. *arXiv* **2024**, arXiv:2401.05561. 927
33. Markov, T.; Zhang, C.; Agarwal, S.; Eloundou, T.; Lee, T.; Adler, S.; Jiang, A.; Weng, L. A holistic approach to undesired content detection in the real world. In *Proceedings of the AAAI Conference on Artificial Intelligence*; 2023; Volume 37, pp. 15009–15018. 928

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content. 929